fire

MDPI

Article

Towards Effective Forest Fire Response: A Cloud–Edge Collaborative UAV Deployment Strategy for Rapid Situational Awareness

Yumin Dong [1,2,3], Peifeng Li [1,3], Xiqing Guo [1,4] and Ziyang Li [1,3,*]

[1] Aerospace Information Research Institute, Chinese Academy of Sciences, Beijing 100094, China
[2] School of Electronic, Electrical and Communication Engineering, University of Chinese Academy of Sciences, Beijing 100049, China
[3] Key Laboratory of Target Cognition and Application Technology (TCAT), Beijing 100190, China
[4] University of Chinese Academy of Sciences, Beijing 100049, China
* Correspondence: zyli@aircas.ac.cn

Abstract

Rapid and balanced situational awareness of fire fronts is critical for effective initial response to forest fires, yet suboptimal task planning for Unmanned Aerial Vehicle (UAV) swarms can delay intelligence delivery. This paper presents a cloud–edge collaborative approach that integrates edge-driven rapid task partitioning with cloud-based global workload balancing, explicitly addressing the NP-hard multiple traveling salesman problem underlying multi-UAV reconnaissance. At the edge, a fire-spread-informed line clustering algorithm quickly assigns monitoring points to UAVs, exploiting low-latency processing for initial sectorization. The cloud then refines this allocation through a novel cooperative–competitive task transfer mechanism that minimizes the makespan. Extensive simulations and a real-world case study based on the 2020 Liangshan wildfire show that the proposed method reduces makespan by up to 24.5% compared to conventional centralized and distributed baselines, while remaining robust under severe communication constraints.

Keywords: forest fire response; UAV; distributed planning; edge computing; load balancing

1. Introduction

The escalating frequency and intensity of forest fires globally demand more agile and effective initial response strategies. Timely and coordinated situational awareness of fire front dynamics is critical for incident commanders to make informed decisions regarding resource deployment, firefighter safety, and containment tactics [1]. As forest fires exhibit rapid spread and complex behavior, agile and adaptive monitoring systems are essential for tracking fire fronts, assessing risk, and guiding resource allocation [2,3]. Unmanned Aerial Vehicles (UAVs), especially when operating in coordinated teams, have emerged as a transformative tool for achieving real-time situational awareness across dynamic and often rugged fire zones, filling a critical gap between satellite imagery and ground reconnaissance [4,5].

Current UAV-based forest fire monitoring primarily adopts three modes: continuous cruising [6,7], dynamic event-driven observation [8], and plan-then-execute observation [9]. While cruising ensures persistent coverage, and dynamic observation enables rapid response, both may suffer from resource inefficiency or coordination difficulties when fire

Academic Editor: Grant Williamson

Received: 12 February 2026
Revised: 8 April 2026
Accepted: 8 April 2026
Published: 10 April 2026

points proliferate. The plan-then-execute observation excels in large-scale scenarios by balancing workloads, minimizing makespan, and avoiding communication overhead. Thus, this paradigm offers a reliable and resource-efficient solution for coordinated multi-UAV reconnaissance in major forest fire emergencies.

However, the observation operational potential of multi-UAV systems is often constrained by a critical management challenge: how to efficiently assign monitoring sectors to each UAV in real-time, ensuring that intelligence is gathered both rapidly and comprehensively across the entire fire front. This task assignment and sequencing challenge is a complex spatial-temporal optimization problem. Inefficient deployment leads to delayed intelligence, uneven coverage, and ultimately, a lengthened operational timeline. Collaborative task planning for UAV swarms involves complex spatial and temporal constraints, forming an NP-Hard combinatorial optimization problem similar to the Multiple Traveling Salesman Problem (MTSP) [10]. The goal is akin to the Multiple Traveling Salesman Problem, where finding the shortest routes for multiple "salesmen" to visit all "cities" (monitoring points) minimizes the overall reconnaissance time.

Traditional solutions follow either centralized or distributed paradigms. Centralized methods, including end-to-end deep learning [11,12] and hierarchical optimization frameworks [12–15], often incur high computational delays. This is primarily because a single central node must process the entire global state and compute plans for all agents, creating a computational bottleneck as the problem scale (number of UAVs and tasks) increases. The resulting latency limits their suitability for real-time emergency scenarios. Distributed approaches, such as auction-based mechanisms [16], reduce computational latency at each node by distributing the computation. However, they rely heavily on continuous, high-frequency communication among all agents to negotiate and reach a consensus on the task allocation. This imposes a significant communication overhead and is often impractical in disaster areas with unstable or limited network bandwidth, where communication links may be intermittent or delayed.

In terms of centralized algorithms, with advancements in neural networks and deep reinforcement learning, end-to-end methods have been proposed; MTSP was transformed into a graph problem, employing computer vision methods and establishing a solution mapping based on convolutional neural networks [17]. Similarly, an end-to-end task allocation algorithm based on deep reinforcement learning for multi-agent autonomous mobile systems was designed [18].

However, given limitations of end-to-end algorithms such as poor interpretability and long computation time, other works utilize phased algorithms for planning. An angle-based grouping strategy coupled with path planning for UAV pesticide coverage operations was proposed, enhancing solution precision [19]. Additionally, the Hungarian algorithm was coupled with an improved D* algorithm for robotic multi-task planning, achieving task allocation and path planning [20].

Given the requirements of high real-time performance and algorithmic interpretability in disaster emergencies and similar contexts, a common approach is to decouple the overarching problem through heuristic algorithms. The core idea of decoupled algorithms involves estimating and predicting task sequences during task allocation prior to path planning. For multi-USV path planning in maritime obstacle environments, a two-stage algorithm was designed: first predicting the number of USVs and allocating tasks based on a greedy strategy, then performing path planning on the allocation result using ant colony optimization [21]. Similarly, a Voronoi-based hybrid ant colony optimization technique for multiple autonomous marine vehicles was proposed, increasing weights in specific regions via Voronoi diagram density [22]. Similarly, for forest fire scenarios, a multi-UAV firefighting task planning method integrating task allocation and path planning was

proposed, considering environmental threats and using improved MP-GWO and NSGA-II algorithms. Simulations showed superior performance over traditional algorithms, with sensitivity analysis highlighting key factors. This study offers an effective solution for multi-UAV task planning in emergencies [23].

In terms of distributed algorithms, distributed methods are increasingly valued for their low latency in time-sensitive scenarios. Multi-robot emergency rescue scenarios were addressed by combining constrained k-means clustering with a consensus-based distributed auction algorithm, solving multi-task allocation and time planning with temporal constraints [24]. Auction mechanisms for UAV swarm task allocation were improved from both functional and mechanistic perspectives, resulting in a proposed two-stage task allocation method based on an improved auction mechanism that yielded favorable experimental results [25]. Distributed algorithms primarily use consensus-based auction methods to maintain consistency among different monitoring platforms [26]. However, these consensus-based auction methods rely on high-overhead real-time communication, potentially compromising algorithm effectiveness in forest fire emergency monitoring planning.

Edge computing offers a viable middle ground by deploying computational resources closer to data sources [27,28]. This architecture supports low-latency processing at the edge while leveraging cloud resources for global coordination. Inspired by this advantage, we propose a cloud–edge collaborative planning method for multi-UAV forest fire monitoring. Our method partitions the planning process into an edge-side task allocation phase and a cloud-side conflict resolution phase, reducing both computational and communication burdens.

Inspired by the above research, this paper proposes a cloud–edge collaborative deployment strategy designed as a decision-support tool for incident management. The main contributions are:

(1) A practical management method that strategically distributes planning tasks between edge devices and a cloud center to balance speed and global coordination.
(2) A fire spread-informed (the fire front propagates in a polygonal manner) task allocation method that rapidly partitions the monitoring area at the edge.
(3) A workload balancing mechanism in the cloud that explicitly minimizes the total time to complete the surveillance mission, a key metric for response effectiveness.
(4) Comprehensive validation demonstrating significant improvements in situational awareness timeliness and system robustness, using both simulations and a real-world fire case study.

The architecture of this paper is shown in Figure 1.

The remainder of this paper is organized as follows: Section 2 formulates the problem. Section 3 details the proposed method. Sections 4 and 5 present experimental results and discussion. Section 6 concludes the paper and suggests future directions.

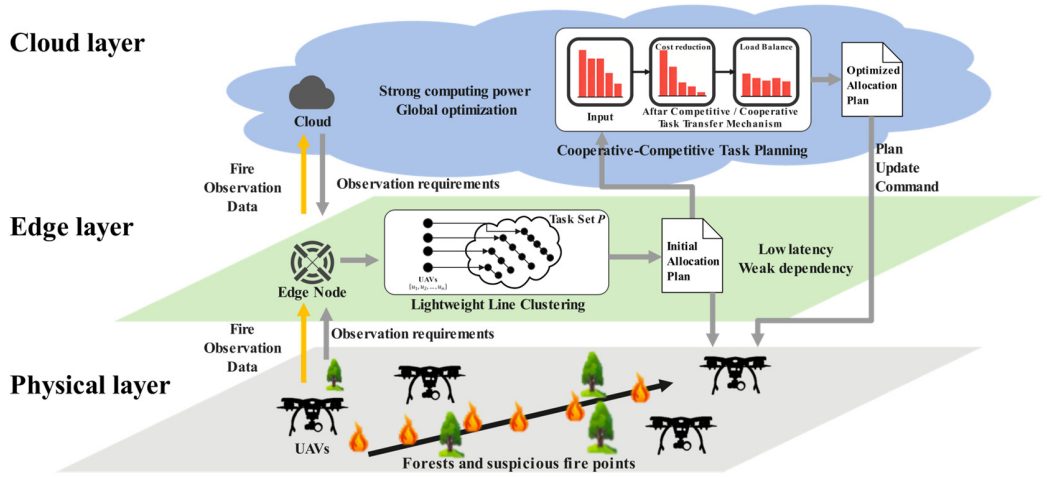

Figure 1. The architecture.

2. Problem Formulation

Based on real-world forest fire data from the literature [29], and considering the critical importance of early-stage monitoring for fire control and the typically scattered nature of early forest fire points, point targets are used to model forest fire monitoring locations. We consider a monitoring system architecture, where a swarm of n UAVs ($\boldsymbol{U}$) collaboratively execute m monitoring tasks ($\boldsymbol{P}$) under the coordination of edge servers and a cloud center, as shown in Equations (1) and (2).

$$\boldsymbol{U} = \{u_1, u_2, \ldots, u_n\}, \quad n \in \boldsymbol{N}^* \tag{1}$$

$$\boldsymbol{P} = \{p_1, p_2, \ldots, p_m\}, \quad m \in \boldsymbol{N}^* \tag{2}$$

The solution is represented as a set of ordered task sequences classified by the UAV set $\boldsymbol{U}$. Given that the number of tasks assigned to each UAV may vary, the solution is not a matrix but a set of vectors, and a solution to the problem is represented by a set of task sequences, one for each UAV, as shown in Equation (3).

$$\mathcal{R} = \{\boldsymbol{r_1}, \boldsymbol{r_2}, \ldots, \boldsymbol{r_n}\} \tag{3}$$

where $\boldsymbol{r_i} = \{p_{i,1}, p_{i,2}, \ldots, p_{i,k_i}\}$ is an ordered sequence of k_i tasks assigned to u_i. The constraints $\bigcup_{i=1}^{n} \mathbf{r_i} = \boldsymbol{P}$ and $\boldsymbol{r_i} \cap \boldsymbol{r_j} = \emptyset, \forall i \neq j$ ensure all tasks are covered exactly once.

The time for u_i to execute its assigned sequence $\mathbf{r_i}$ is calculated as the total flight time from its initial position, through all tasks in order, and back to its start point:

$$\mathbf{T}(\mathbf{r_i}) = \frac{\|x_i - p_{i,1}\|}{v_i} + \sum_{j=1}^{k_i-1} \frac{\|p_{i,j} - p_{i,j+1}\|}{v_i} + \frac{\|p_{i,k_i} - x_i\|}{v_i} \tag{4}$$

where $\| \ \|$ denotes the Euclidean distance; x_i is the initial position of u_i; v_i is the flying speed of u_i; and u_i departs from x_i, completes all monitoring tasks, and returns to x_i. The overall system execution time vector is $\boldsymbol{T}(\mathcal{R}) = (\boldsymbol{T}(\boldsymbol{r_1}), \boldsymbol{T}(\boldsymbol{r_2}), \ldots, \boldsymbol{T}(\boldsymbol{r_n}))$.

Existing work primarily focuses on two optimization objectives:

(1) Minimizing the total monitoring time;
(2) Minimizing the maximum monitoring time of any single UAV.

The former focuses on optimizing the cost per task, while the latter focuses on optimizing the workload balance across monitoring platforms; these objectives are sometimes inconsistent. Considering that the overall monitoring time of a distributed cooperative monitoring system is determined by the longest execution time among its subsystems,

the optimization objective of this paper is defined as minimizing the maximum UAV monitoring time.

A set of point-based monitoring targets (e.g., fire ignition points, critical monitoring locations) distributed across a geographic area is considered. The system is composed of n UAVs and m tasks, as defined in Equations (1) and (2).

The objective is to minimize the maximum mission time of any UAV, as defined in Equation (5), so that load balancing and timely completion are ensured.

$$\begin{gathered}\mathcal{R}^* = \arg\min(\max(\boldsymbol{T}(\mathcal{R}))) \\ s.t. \bigcup_{i=1}^{n} \boldsymbol{r_i} = \boldsymbol{P}, \boldsymbol{r_i} \cap \boldsymbol{r_j} = \emptyset, (\forall i \neq j) \\ \|p_i - p_j\| = \|p_j - p_i\|, (\forall i \neq j) \\ \forall k \in \{1, \ldots, m-1\}, Inf > \|p_k - p_{k+1}\| > 0 \\ \forall i,\ v_i = v \\ \forall E(\boldsymbol{T}(\boldsymbol{r}_i)) = \alpha \times \boldsymbol{T}(\boldsymbol{r}_i) + \beta \times E_{task} \leq E_i\end{gathered} \tag{5}$$

The optimization goal is to find a task allocation $\mathcal{R}^*$ such that $\boldsymbol{T}(\mathcal{R})$ is minimized. Constraints are: all target points must be visited exactly once by exactly one UAV; paths between monitoring targets are undirected; considering that the flight area over forests is usually vast and undisturbed, all target points are dispersed but connected (i.e., the graph is connected); no-fly zones and physical obstacles are not considered; and UAVs fly at a constant cruising speed v. Energy consumption is modeled in $E(\boldsymbol{T}(\boldsymbol{r}_i))$, where α and β are platform-specific coefficients; the mission for each UAV must not exceed its battery capacity E_i, which is set conservatively to ensure energy feasibility. In the current evaluation, E_i is set to be sufficiently large to ensure all generated plans are feasible, focusing on the relative balance of workload. This constraint is explicitly implemented in our algorithm and can be tightened for specific UAV models in practice.

While these constraints and assumptions define the core optimization problem, they carry important implications for real-world system deployment. The limited communication range necessitates robust network strategies, such as deploying mobile edge relays or employing UAVs as communication bridges, to maintain connectivity in complex terrains. The assumption of static, independent tasks implies that our method in its current form operates in a planning-then-execution mode; for continuous monitoring of dynamically spreading fires, this would require periodic re-invocation of the method with updated task lists, integrating real-time fire spread prediction models. Omitting physical obstacles simplifies the path cost calculation but is addressed in practice by layering a local obstacle-avoidance path planner upon the task sequences generated by our method. Finally, the homogeneous UAV model and simplified energy model provide a clear baseline; extending to heterogeneous platforms would involve modifying the cost functions in the conflict resolution stage to account for differing capabilities, and the conservative battery capacity would be dynamically adjusted based on real-time battery status and precise energy consumption profiles—both are straightforward extensions within our modular architecture. Thus, these assumptions do not preclude real-world application but rather define a clear scope and pathway for integration with complementary subsystems.

3. The Proposed Cloud–Edge Collaborative Planning Method

Figure 2 illustrates the architecture of the proposed planning method and decouples the NP-Hard problem into two manageable stages: (1) Edge-side Task Allocation and (2) Cloud-side Conflict Resolution, include competitive task transfer and cooperative task

transfer. This structure leverages the low-latency processing of edge nodes for rapid initial allocation and the powerful computational resources of the cloud for global optimization.

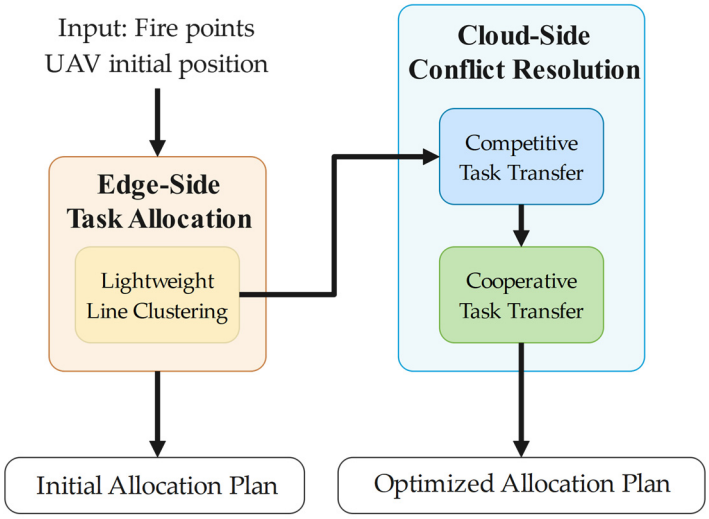

Figure 2. Workflow of the proposed cloud–edge collaborative planning method.

3.1. Task Allocation Stage

The first stage is deployed on the edge server, functioning as a local controller to quickly partition the task set $\boldsymbol{P}$ among the UAVs $\boldsymbol{U}$, producing an initial allocation $\mathcal{R}^{(0)}$.

Common approaches to balance solution quality and algorithm runtime include greedy algorithms, Voronoi diagrams, and k-means clustering. Greedy algorithms consider the UAV's path step-by-step, equivalent to rapidly selecting a path for the UAV based on current distance calculations, but are prone to local optima. Voronoi-based algorithms primarily rely on the distance between UAV starting positions and targets for spatial partitioning and rapid task allocation, but only consider initial positions and require uniform target distribution. K-means clustering algorithms group targets to ensure proximity within clusters but fail to account for the often-distant initial UAV positions and ignore the relevance of points along the path.

Therefore, this paper integrates and improves upon these ideas. Considering the characteristics of forest fire spread [30], the cluster centroid is changed from a point to a line segment. This line segment estimates the UAV path, and targets are classified based on their distance to this segment. The segment is iteratively updated until convergence. The specific algorithm flow is shown in Algorithm 1.

The time complexity per iteration is $O(m \cdot n)$. Thus, the total time complexity is $O(iter \cdot m \cdot n)$, where iter is the number of iterations, matching the complexity of standard k-means. Given its relatively low computational complexity, this process is highly suitable for resource-constrained edge nodes. The algorithm converges as the objective function (sum of squared distances to segments) is non-negative and monotonically decreasing with each assignment and update step; the following is a proof of convergence:

Define Algorithm 1 objective function $\mathcal{J}$ as the sum of squared distances from all task points p to their assigned UAVs' line $\boldsymbol{l}_i$ segments.

$$\mathcal{J} = \sum_{i \in \boldsymbol{U}} \sum_{p \in \boldsymbol{r}_i} \|\boldsymbol{p}, \boldsymbol{l}_i\|^2 \tag{6}$$

$\mathcal{J}$ is only changed during the allocation process, and

$$\left\|p_j, \boldsymbol{l}^*\right\|^2 < \left\|p_j, \boldsymbol{l}_i\right\|^2 \ \forall i \in \boldsymbol{U} \tag{7}$$

Equation (7) defines the assignment rule for each task point p_j during each iteration of the optimization process. Specifically, for a given point p_j, let $\boldsymbol{l}^*$ denote the line segment associated with a particular UAV that yields the smallest squared distance to p_j among all UAVs. The condition states that p_j is assigned to $\boldsymbol{l}^*$ if and only if $\left\|p_j, \boldsymbol{l}^*\right\|^2 < \left\|p_j, \boldsymbol{l_i}\right\|^2$ for every other UAV segment $\boldsymbol{l_i}$.

Algorithm 1: Task allocation based on line clustering

Input: $\boldsymbol{U} = \{\boldsymbol{u_1}, \boldsymbol{u_2}, \ldots, \boldsymbol{u_n}\}$, with initial positions $\{x_1, x_2, \ldots, x_n\}$
$\boldsymbol{P} = \{p_1, p_2, \ldots, p_m\}$
Output: $\mathcal{R} = \{\boldsymbol{r_1}, \boldsymbol{r_2}, \ldots, \boldsymbol{r_n}\}$

for each UAV $u_i \in \boldsymbol{U}$ **do**
  Initialize its cluster line segment $\boldsymbol{l_i}$ with endpoints (ep_{1^i}, ep_{2^i}):
  $ep_{1^i} \leftarrow x_i$
  $ep_{2^i} \leftarrow \arg\min\|x_i - p\|, \quad p \in \boldsymbol{P}$
  Initialize task sequence $\boldsymbol{r_i} \leftarrow \emptyset$
end for
while not converged **do**
  for each task $p_j \in \boldsymbol{P}$ **do**
    Find UAV u^* whose segment $\boldsymbol{l}^*$ minimizes $\left(p_j, \boldsymbol{l_i}\right)$
    Assign p_j to $\boldsymbol{r}^*$
  end for
  for each UAV $u_i \in \boldsymbol{U}$ **do**
    if $\boldsymbol{r_i}$ is not empty **then**
      Update the segment by moving endpoint ep_{2^i} to the centroid of $\boldsymbol{r_i}$
    end if
  end for
end while
return $\mathcal{R}$

The sequence of objective function values $\mathcal{J}^{(1)}, \mathcal{J}^{(2)}, \mathcal{J}^{(3)}, \ldots$ generated after each iteration is monotonically decreasing. Since the sum of squared distances $\mathcal{J} \geq 0$, the sequence is also bounded. By the Monotone Convergence Theorem, the sequence $\mathcal{J}^{(iter)}$ must converge.

Upon convergence, a heuristic algorithm sequences the task order for each UAV. Ant Colony Optimization (ACO) [31] mimics the process of ants finding food paths. Given that emergency monitoring task sequencing is a path planning problem, ACO is used for task sequence ordering within each UAV's assigned cluster.

3.2. Conflict Resolution Stage

In this stage, the cloud center aggregates the initial allocation plans from multiple edge nodes to perform global optimization, ensuring system-wide quality of service (QoS). This reflects the typical cloud-side role in edge computing architecture for macro-level coordination. This paper adopts a boundary-effect-based importance metric proposed in the literature [32], where each task's significance is measured by its contribution to the local cost generated by a vehicle. In our context, the importance metric of a task is defined as the bidirectional cost differential for a UAV between performing and not performing that task, capturing both the task's direct cost and its synergistic effects on other tasks in the sequence. While this metric effectively reduces the insertion/removal cost of individual tasks, it may lead to imbalanced task load distribution when the objective is min–max cost.

Therefore, this paper builds upon it, designing a cooperative–competitive mechanism for task transfer. In competitive task transfer phase, each monitoring platform auctions tasks based on cost to minimize local task cost. And in cooperative task transfer phase, tasks are transferred from UAVs with longer execution times to those with shorter times, aiming to optimize the global maximum monitoring time (workload balancing).

3.2.1. Competitive Task Transfer Mechanism

The impact of moving a task p from r_i to r_j is evaluated using the following:
Insertion Cost:

$$W^{\oplus}(r_j,\ p) = \min_z \{T(r_j \oplus_z p)\} - T(r_j) \tag{8}$$

Removal Gain:

$$W^{\ominus}(r_i,\ p) = T(r_i) - T(r_i \ominus p) \tag{9}$$

$W^{\oplus}(r_j, p)$ represents the minimum cost increment caused by inserting task p into task sequence r_j; $\oplus_z$ denotes inserting the task at position z in the queue.

$W^{\ominus}(r_i, p)$ represents the maximum cost reduction caused by removing task p from task sequence r_i; $\ominus$ denotes removing the specified task from the queue. (z is implicit in finding the best removal impact).

The profit $B(r_i, r_j, p)$ of transferring task p from task r_i to r_j is defined as

$$B(r_i, r_j, p) = W^{\ominus}(r_i, p) - W^{\oplus}(r_j, p) \tag{10}$$

In the competitive phase, the insertion/removal importance is calculated for each task point p and each UAV queue r_i. A task transfer is executed only if it reduces the total system cost (i.e., the sum of the individual UAV execution times). The specific algorithm flow is shown in Algorithm 2.

Algorithm 2: Competitive Task Transfer Mechanism

Input: Task sequence $\mathcal{R} = \{r_1, r_2, \ldots, r_n\}$
Output: Improve task sequence $\mathcal{R}^* = \{r_1^*, r_2^*, \ldots, r_n^*\}$ and importance metric matrix $W = \{W^{\oplus}, W^{\ominus}\}$

do
  for each task $p \in P$ and each task pair $(r_i, r_j), i \neq j$ **do**
    Calculate profit: $\mathbf{B}(r_i, r_j, p) = \mathbf{W}^{\ominus}(r_i, p) - \mathbf{W}^{\oplus}(r_j, p)$
  end for
  Find $B(r_i^*, r_j^*, p^*) = \arg\max B(r_i, r_j, p)$
  if $B(r_i^*, r_j^*, p^*) > 0$ **then**
    Execute transfer
    $r_i^* \leftarrow r_i \ominus p$
    $r_j^* \leftarrow r_j \oplus_z p$
  end if
while $\max_{(i,j,p)} B(r_i, r_j, p) > 0$
$\mathcal{R}^* \leftarrow \{r_1, r_2, \ldots, r_n\}$
return $\mathcal{R}^*$

The algorithm iterates over each task $p \in P$ ($O(m)$) and each unordered task pair $(r_i, r_j), i \neq j$. The number of task pairs is $O(n^2)$. For each triple (r_i, r_j, p), it calculates $B(r_i, r_j, p) = W^{\ominus}(r_i, p) - W^{\oplus}(r_j, p)$; calculating $W^{\oplus}(r_j, p)$ requires finding the optimal insertion position z^* in sequence r_j, which typically involves evaluating insertion at all possible $|r_j| + 1$ positions; in the worst case, $|r_j| = m$, making this calculation $O(m)$, and

calculating $\boldsymbol{W}^{\ominus}(\boldsymbol{r_i}, p)$ is $O(1)$ due to the removal cost is stored. So, the total complexity for the profit calculation step is $O\left(m^2 \cdot n^2\right)$.

Finding the maximum value in a structure has complexity $O\left(m \cdot n^2\right)$. Removing a task from one sequence and inserting it into another is an $O(m)$ operation.

Let iter be the number of iterations until the algorithm converges. The overall worst-case time complexity of Algorithm 2 is

$$O(iter) \cdot \left[O\left(m^2 \cdot n^2\right) + O\left(m \cdot n^2\right) + O(m)\right] = O\left(iter \cdot m^2 \cdot n^2\right) \tag{11}$$

Define the objective function for this phase as the total cost of the system:

$$C_{\text{total}} = \sum_{i=1}^{n} \boldsymbol{T}(\mathbf{r_i}) \tag{12}$$

The competitive transfer mechanism aims to minimize C_{total}.

This represents the net reduction in the C_{total} if the transfer is executed:

$$\Delta C_{\text{total}} = B\left(\boldsymbol{r_i}^*, \boldsymbol{r_j}^*, p^*\right) > 0 \tag{13}$$

A transfer is only executed if $B\left(\boldsymbol{r_i}^*, \boldsymbol{r_j}^*, p^*\right) > 0$, meaning each successful transfer strictly decreases the C_{total}. The sequence of $C_{\text{total}}^{(1)}, C_{\text{total}}^{(2)}, C_{\text{total}}^{(3)}, \ldots$ generated after each transfer is strictly monotonically decreasing.

The C_{total} is bounded below. The sum of the distances between all tasks and the sum of the distances from each UAV's start point to its first task and back from its last task constitute a lower bound.

Since the sequence $C_{\text{total}}^{(k)}$ is monotonically decreasing and bounded below, it must converge, by the Monotone Convergence Theorem.

3.2.2. Cooperative Task Transfer Mechanism

After competitive transfer, individual task costs are optimized, but boundary effects may cause some UAVs to get overloaded. According to Equation (5), the objective is to minimize the maximum UAV time costs. Therefore, after reducing task costs, the cooperative task transfer mechanism is proposed to balance the monitoring load across UAVs.

Building upon the importance metrics, a cooperative gain importance metric $\boldsymbol{W_{coop}}$ is designed by Equation (14).

$$\boldsymbol{W_{coop}}\left(\boldsymbol{r_i}, \boldsymbol{r_j}, p\right) = \boldsymbol{W}^{\ominus}(\boldsymbol{r_i}, p) - \boldsymbol{W}^{\oplus}\left(\boldsymbol{r_j}, p\right) + \left(\max(\boldsymbol{T}(\mathcal{R})) - \boldsymbol{T}\left(\boldsymbol{r_j}\right)\right) \tag{14}$$

$\boldsymbol{W_{coop}}$ considers not only the cost change from the transfer itself $(\boldsymbol{W}^{\ominus}(\boldsymbol{r_i}, p) - \boldsymbol{W}^{\oplus}(\boldsymbol{r_j}, p))$ but also introduces a load balancing term $(\max(\boldsymbol{T}(\mathcal{R})) - \boldsymbol{T}(\boldsymbol{r_j}))$. This encourages UAV u_j whose current load $(\boldsymbol{T}(\boldsymbol{r_j}))$ is below the system maximum load $(\max(\boldsymbol{T}(\mathcal{R})))$ to be more inclined to receive task p, even if it causes a slightly higher cost increase $(\boldsymbol{W}^{\oplus}(\boldsymbol{r_j}, p))$ for u_j itself. The rationale is that this helps reduce the system's maximum load $(\max(\boldsymbol{T}(\mathcal{R})))$. The core idea of this design is to prioritize reducing the peak load of the system within a controllable increase in overall cost.

The specific algorithm flow is shown in Algorithm 3.

The time complexity of Algorithm 3 is analyzed per iteration of its main loop.

Finding the UAV with maximum execution time requires comparing all n UAVs' complexity $O(n)$. The algorithm iterates over each task p in the sequence of the most overloaded UAV and each other UAV. In the worst case, the overloaded UAV could have m tasks. For each such task and each of the $n-1$ other UAVs, it calculates the cooperative gain $\boldsymbol{W_{coop}}\left(\boldsymbol{r_{i_{max}}}, \boldsymbol{r_j}, p\right)$.

Algorithm 3: Cooperative Task Transfer Mechanism

Input: $\mathcal{R}^* = \{r_1{}^*, r_2{}^*, \ldots, r_n{}^*\}$
Output: $\mathcal{R}^{**} = \{r_1{}^{**}, r_2{}^{**}, \ldots, r_n{}^{**}\}$

do
  Find most overloaded UAV: $i_{\max} \leftarrow \underset{i}{\operatorname{argmax}}\, \boldsymbol{T}(\boldsymbol{r_i})$
  for each task $p \in \boldsymbol{r}_{i_{max}}$ and each $u_j, j \neq i_{\max}$ **do**
    Calculate cooperative gain:
    $\boldsymbol{W_{coop}}\left(\boldsymbol{r}_{i_{\max}}, \boldsymbol{r_j}, \mathrm{p}\right) = \boldsymbol{W}^{\ominus}(\boldsymbol{r}_{i_{\max}}, p) - \boldsymbol{W}^{\oplus}(\boldsymbol{r_j}, p) + \left(\boldsymbol{T}(\boldsymbol{r}_{i_{\max}}) - \boldsymbol{T}(\boldsymbol{r_j})\right)$
  end for
  Find the most beneficial transfer: $\boldsymbol{W_{coop}}\left(\boldsymbol{r}_{i_{\max}}{}^*, \boldsymbol{r_j}{}^*, p^*\right) =$
  $\arg\max \boldsymbol{W_{coop}}\left(\boldsymbol{r}_{i_{\max}}{}^*, \boldsymbol{r_j}, p\right)$
  if $\boldsymbol{W_{coop}}\left(\boldsymbol{r}_{i_{\max}}{}^*, \boldsymbol{r_j}{}^*, p^*\right) > 0$ and $\boldsymbol{T}(\boldsymbol{r}_{i_{\max}}) > \boldsymbol{T}(\boldsymbol{r_j}{}^*)$ **then**
    Execute transfer:
    $\boldsymbol{r}_{i_{max}}{}^* \leftarrow \boldsymbol{r}_{i_{max}} \ominus p^*$
    $\boldsymbol{r_j}{}^* \leftarrow \boldsymbol{r_j} \oplus_z p^*$
  end if
while $\max\limits_{(i,j,p)} \boldsymbol{W_{coop}}(\boldsymbol{r}_{i_{\max}}, \boldsymbol{r_j}, p) > 0$
$\mathcal{R}^{**} \leftarrow \{r_1{}^*, r_2{}^*, \ldots, r_n{}^*\}$
return $\mathcal{R}^{**}$

Calculating $\boldsymbol{W}^{\oplus}(\boldsymbol{r_j}, p)$ requires finding the optimal insertion position in sequence $\boldsymbol{r_j}$, which is $O(|\boldsymbol{r_j}|) = O(m)$. Calculating $\boldsymbol{W}^{\ominus}(\boldsymbol{r}_{i_{\max}}, p)$ is $O(1)$. Accessing $\boldsymbol{T}(\boldsymbol{r}_{i_{\max}})$ and $\boldsymbol{T}(\boldsymbol{r_j})$ is also $O(1)$.

Finding the maximum value among $O(m \cdot n)$, gain values have complexity $O(m \cdot n)$. Removing a task from one sequence and inserting it into another is an $O(m)$ operation.

Let iter be the number of iterations until the algorithm converges. The overall worst-case time complexity of Algorithm 3 is

$$O(iter) \cdot \left[O(n) + O\left(m^2 \cdot n\right) + O(m \cdot n) + O(m)\right] = O\left(iter \cdot m^2 \cdot n\right) \tag{15}$$

The convergence of Algorithm 3 is proven by analyzing its effect on the maximum UAV cost:

$$C_{\max} = \max_i \boldsymbol{T}(\boldsymbol{r_i}) \tag{16}$$

The cooperative transfer mechanism explicitly aims to minimize the $C_{\max}$.

The cooperative gain $\boldsymbol{W_{coop}}\left(\boldsymbol{r}_{i_{\max}}, \boldsymbol{r_j}, p\right)$ of transferring task p from the most overloaded task $\boldsymbol{r}_{i_{\max}}$ to $\boldsymbol{r_j}$ is designed to capture the potential improvement in the maximum cost.

The term $\boldsymbol{W}^{\ominus}(\boldsymbol{r}_{i_{\max}}, p) - \boldsymbol{W}^{\oplus}(\boldsymbol{r_j}, p)$ represents the net change in the sum of costs. The term $\left(\boldsymbol{T}(\boldsymbol{r}_{i_{\max}}) - \boldsymbol{T}(\boldsymbol{r_j})\right)$ encourages load balancing by favoring transfers to less loaded UAVs. A transfer is only executed if $max\ \boldsymbol{W_{coop}}\left(\boldsymbol{r}_{i_{\max}}, \boldsymbol{r_j}, p\right) > 0$ and $\boldsymbol{T}(\boldsymbol{r}_{i_{\max}}) > \boldsymbol{T}(\boldsymbol{r_j}{}^*)$.

The sequence of $C_{\max}^{(1)}, C_{\max}^{(2)}, C_{\max}^{(3)}, \ldots$ generated after each transfer is strictly monotonically decreasing.

The $C_{\max}$ is bounded below ($C_{\max} > 0$). Since the sequence $C_{\max}^{(k)}$ is monotonically decreasing and bounded below, it must converge, by the Monotone Convergence Theorem.

4. Simulation Experiments

To evaluate the performance of the proposed planning method, this chapter conduct experiments using both simulated analog tasks and real-world forest fire data. The simulation-based experiments are divided into two parts: (i) scalability analysis under uniformly random task distributions with varying numbers of tasks and UAVs, and (ii) robustness evaluation under different task generation patterns (uniform random, clustered, and linear) to assess sensitivity to fire front geometry. For the real-world case study based on the Liangshan wildfire, we first compare the proposed method against benchmark algorithms, and then perform an ablation study to quantify the contribution of each component.

Prior to experimental evaluation, three representative multi-task allocation methods were selected as benchmarks to comprehensively evaluate the performance of the proposed approach. These methods were chosen based on distinct optimization mechanisms, covering centralized, distributed, and spatial partitioning strategies, ensuring strong representativeness:

(1) A phased centralized method using greedy allocation and ACO path planning [21], valued for speed but prone to local optima.
(2) A Voronoi-based spatial partitioning method [22], intuitive but sensitive to initial conditions.
(3) A distributed K-means and auction-based method [24], adaptable but communication-dependent.

We exclude deep learning methods due to their lack of interpretability, high data needs, and computational cost, which are critical constraints in emergency response.

4.1. Simulation Experiments on Generation Tasks

To evaluate the performance of the proposed planning method under various conditions, we first conducted simulations based on generated tasks; to simulate different spatial distributions of tasks, three distinct generation modes were employed, as shown in Figure 3:

(1) Uniform random: Task locations are drawn uniformly from the entire grid. It serves as a baseline, representing scattered ignition points without spatial structure.
(2) Clustered: Task points are grouped into 3–5 clusters. Within each cluster, points are generated by adding Gaussian noise to a randomly chosen cluster center. It simulates multiple fire heads or spot fires, testing performance when tasks are concentrated in distinct regions.
(3) Linear: Task points are predominantly distributed near the outline of a randomly generated polygon, while a small fraction is placed inside the polygon. It approximates a linear fire front, evaluating the algorithm under its most favorable condition.

4.1.1. Simulation Based on Uniform Random Tasks

The primary scalability experiments are conducted under the uniform random mode because it represents the most general and unbiased scenario for initial validation. In real-world forest fire monitoring, early-stage ignition points can appear arbitrarily across the terrain without a predefined spatial structure. Evaluating the proposed method under this baseline condition ensures that its performance is not artificially inflated by a favorable fire geometry and demonstrates its applicability to a wide range of unforeseen situations. Moreover, uniform random distributions are commonly adopted in the literature for multi-UAV task allocation studies, facilitating fair comparisons with existing benchmarks. The robustness to other fire patterns (clustered and linear) is subsequently assessed in dedicated experiments, providing a comprehensive view of the algorithm's strengths and limitations.

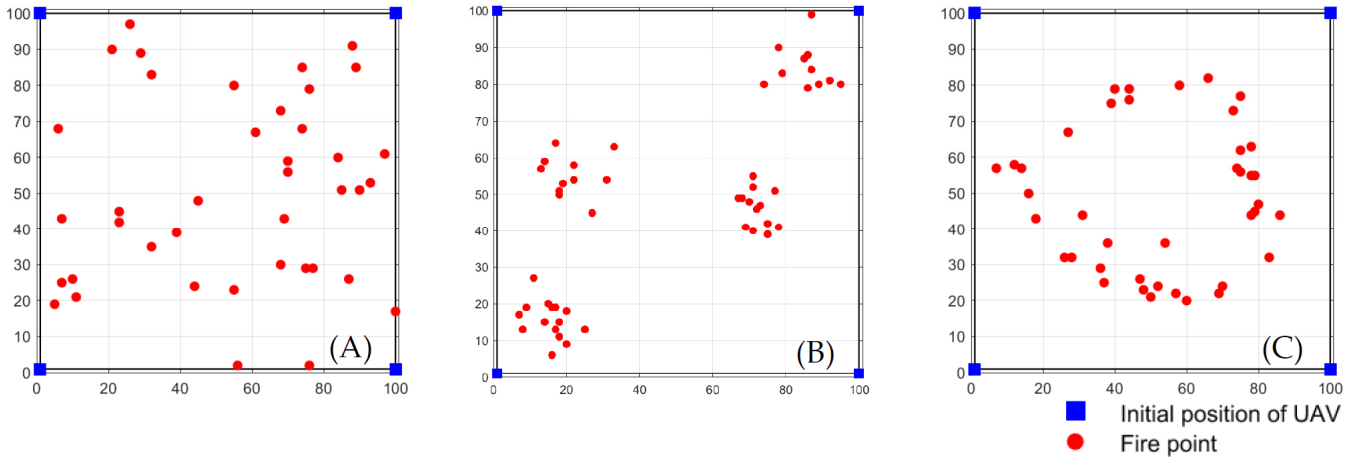

Figure 3. Example of three distinct generation modes: (**A**) uniform random (**B**) clustered (**C**) linear.

We generate 12 scenarios with tasks (as Table 1 shown), and UAVs randomly distributed in a 10 km × 10 km area were generated. Task locations were sampled from a uniform distribution. UAV start positions were set at the boundaries of the area. Each configuration is run 50 times independently with the same task and UAV positions to evaluate the stability of the algorithm—i.e., its consistency under identical fire points, where only the internal randomness of the optimization process changes.

Table 1. Simulation scenario setting.

	1	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
Number of tasks	20	30	40	50	40	50	60	70	60	70	80	90
Number of UAVs	3	3	3	3	4	4	4	4	5	5	5	5

Algorithm runtime and optimization results are shown in Tables 2–4.

Table 2. Avg. task computation time in simulated scenarios/s.

Scenario	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
Greedy based	**1.2324**	**2.1687**	2.9967	4.1656	2.9659	3.939	5.1145	6.8632	5.2113	6.9246	8.4968	**9.7546**
Voronoi Diagram based	1.2727	2.1888	2.9143	4.4271	2.9688	3.9264	5.1586	**6.487**	**5.0314**	6.5731	**8.0859**	10.2183
K-means Clustering based	1.2672	2.1734	**2.8875**	**4.1243**	2.9558	**3.9067**	**5.0565**	6.5108	5.0612	**6.5675**	8.117	10.3828
Proposed method	1.2743	2.1753	2.9157	4.312	**2.8873**	3.9169	5.1365	6.5611	5.0591	6.8891	8.1955	10.3175

Bold denotes the best result for each metric.

Table 3. Total monitoring cost score for simulated scenarios.

Scenario	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
Greedy based	**622.6375**	946.8436	945.0274	1094.9309	980.0441	1031.5128	1164.3533	1417.4874	1459.589	1523.0577	1537.0129	1793.4573
Voronoi Diagram based	624.5946	**728.7009**	884.9268	**818.7202**	**805.724**	**805.7183**	**886.2025**	1094.418	1004.5162	1025.5709	1105.1264	1132.7282
K-means Clustering based	624.5946	**728.7009**	859.8789	828.2596	**805.724**	815.4666	904.2831	**1079.6862**	943.9281	1021.9563	**1103.2562**	**1122.2915**
Proposed method	624.5946	734.2715	**847.2917**	842.4112	816.6927	824.432	905.089	1093.389	**943.749**	**1010.437**	1087.2915	1139.5069

Bold denotes the best result for each metric.

Table 4. Maximum unmanned aerial vehicle monitoring cost score in simulated scenarios.

Scenario	1	2	3	4	5	6	7	8	9	10	11	12
Greedy based	**195.0571**	251.7833	366.044	348.6386	269.8394	352.0564	396.5114	575.1459	402.9016	513.705	499.4251	525.3304
Voronoi Diagram based	225.8805	213.6698	256.1198	251.3437	272.8539	218.7673	248.5645	320.1574	289.2714	275.0055	307.4966	323.2703
K-means Clustering based	225.8805	213.6698	223.626	**219.7545**	272.8539	218.7673	248.5645	288.1178	**248.4546**	269.1306	312.171	351.4983
Proposed method	225.8805	**202.2265**	**222.6087**	219.7566	**246.0706**	**218.206**	**244.3277**	**286.724**	**248.4546**	**264.1549**	**298.6199**	**300.9793**

Bold denotes the best result for each metric.

Tables 2–4 summarize the performance of all four algorithms across the 12 simulated scenarios described in Table 1. Table 2 reports the average computation time (in seconds), demonstrating the computational efficiency of each method. Table 3 presents the total travel distance (sum of all UAV flight distances), which reflects the overall resource consumption. Table 4 shows the makespan (maximum UAV flight distance), the primary optimization objective of this study, indicating the workload balance among UAVs.

4.1.2. Robustness Evaluation Under Different Task Distributions

To assess the sensitivity of the proposed method to fire front geometry, we fix the number of UAVs to four and tasks to 50, and generate task sets using each of the three modes (uniform random, clustered, linear). Each configuration is run 50 times independently to evaluate the robustness of the algorithm—i.e., its ability to maintain performance across diverse spatial patterns of fire points.

Table 5 summarizes the monitoring planning results for all algorithms under the three distributions. The proposed method achieves the lowest makespan in the all three modes, confirming its design advantage.

Table 5. Monitoring planning results under different task distributions.

Algorithm	Avg. Computation Time (s)			Avg. Total Cost Score			Avg. Max UAV Cost Score		
	Random	Clustered	Linear	Random	Clustered	Linear	Random	Clustered	Linear
Greedy based	4.0567	4.0533	4.3991	1496.6357	690.6502	1036.4805	486.0374	357.7440	449.3586
Voronoi Diagram based	**3.9724**	4.0596	4.0190	1127.1659	763.6564	983.2095	319.4863	289.8564	294.4731
K-means Clustering based	3.9799	4.0710	3.9795	**1106.9776**	**690.0091**	**957.8901**	321.2330	279.7222	289.4738
Proposed method	4.0157	**4.0485**	**3.9137**	1108.9802	865.4131	1005.5175	**312.1384**	**266.2325**	**276.6230**

Bold denotes the best result for each metric.

4.2. Simulation Based on Real Forest Fire Emergency Monitoring Task

Furthermore, to validate the method's practicality in a real-world forest fire scenario, we employed data from an actual forest fire event, using data from a forest fire occurring on 7 May 2020, in Liangshan, Sichuan, China [29], with the fire center at 28°14′ N, 102°14′ E. Sampled fire point data was used. Four UAVs were set to observe from four different directions ((20,28), (50,28), (20,80), (50,80)). The distribution of fire points is shown in Figure 4, and the planning result is shown in Figure 5.

Simulation results using real Liangshan forest fire data (50 runs) are shown in Table 6.

4.3. Ablation Study on Real Forest Fire Emergency Monitoring Task

To thoroughly evaluate the individual contribution of each key component within the proposed method, an ablation study was conducted using the same real-world Liangshan forest fire data as described in Section 4.2. The study compares four distinct experimental configurations; each run 50 times independently to obtain average performance metrics. The configurations are designed to isolate the effects of the core algorithmic stages.

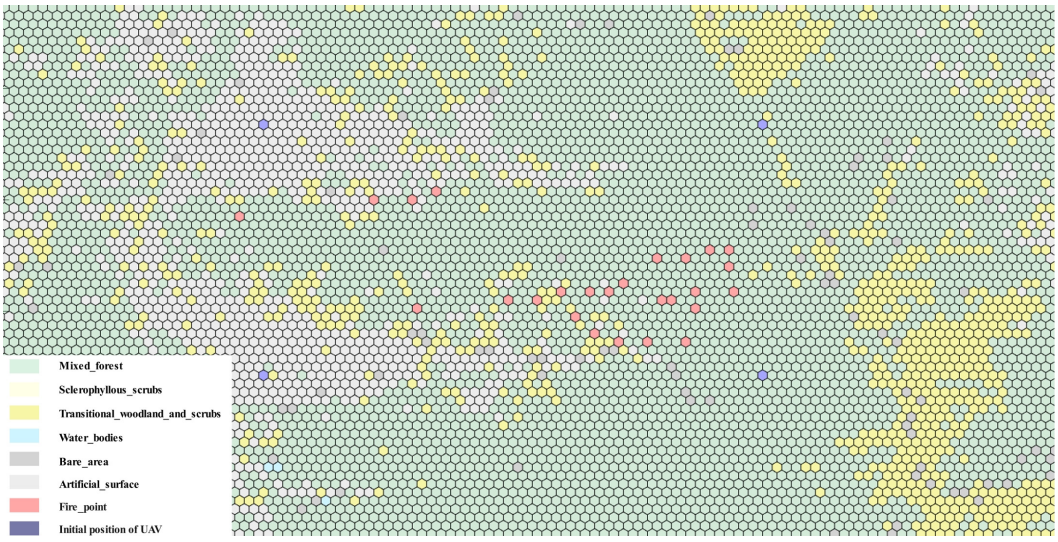

Figure 4. The distribution of fire points.

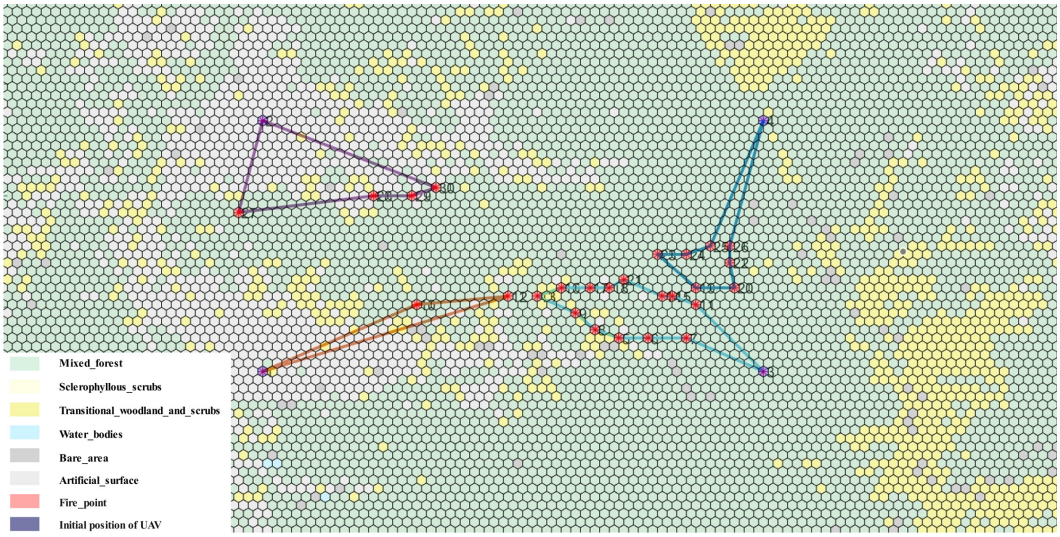

Figure 5. The planning result.

Table 6. Liangshan forest fire monitoring planning results.

Algorithm	Avg. Computation Time (s)	Avg. Total Cost Score	Avg. Max UAV Cost Score
Greedy based	1.9670	316.7338	147.0641
Voronoi Diagram based	2.2706	302.8349	123.5188
K-means Clustering based	2.2609	**302.3677**	123.0516
Proposed method	**1.7330**	351.6118	**92.9065**

Bold denotes the best result for each metric.

Random Allocation: Serves as the baseline, where monitoring tasks are assigned to UAVs randomly, without any optimization strategy. This tests the worst-case scenario.

Random + Cooperative–Competitive: Utilizes the same random initial task allocation as Configuration 1 but subsequently applies the proposed cooperative–competitive conflict resolution mechanism in the cloud. This tests the effectiveness of the conflict resolution mechanism alone when starting from a very poor initial allocation.

Line Clustering: Employs only the proposed line clustering algorithm for task allocation at the edge but omits the subsequent cloud-based cooperative–competitive con-

flict resolution stage. This tests the effectiveness of the edge-side allocation algorithm in isolation.

Full Method: Represents the complete proposed method, integrating both the edge-based line clustering task allocation and the cloud-based cooperative–competitive conflict resolution mechanism.

The results of the ablation study are summarized in Table 7 and Figure 6.

Table 7. Results of the ablation study on the real forest fire case.

No.	Experiment Category	Avg. Computation Time (s)	Avg. Total Cost Score	Avg. Max UAV Cost Score
1	Random Allocation	**1.151**	706.0664	207.9541
2	Random + Cooperative–Competitive	1.428	593.9401	186.4340
3	Line Clustering	1.389	354.8634	103.0944
4	Full Method	1.733	**351.6118**	**92.9065**

Bold denotes the best result for each metric.

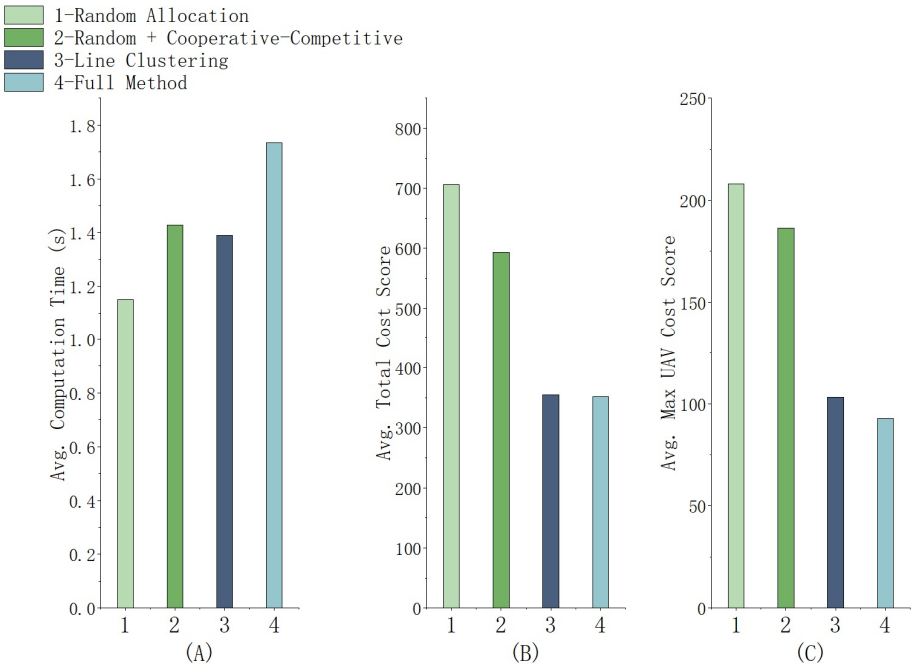

Figure 6. Ablation study results for the real forest fire scenario: (**A**) avg. computation time, (**B**) avg. total cost score, (**C**) avg. max UAV cost score.

5. Discussion

5.1. Analysis of Generation Task Simulation Results

5.1.1. Analysis of Uniform Random Tasks

(1) Analysis of Computation Time

The results indicate that the proposed method maintains stable computational times across a wide range of problem scales, avoiding the exponential growth characteristic of exact optimization solvers. Their performance is expected, as both rely on spatial partitioning with linear complexity.

In comparison to these approaches, the proposed method maintains stable and competitive computation times across all 12 scenarios, with a maximum of only 10.32 s for the largest instance. This stability is a direct result of its core architecture: the linear complexity of the edge-side line clustering algorithm effectively avoids the exponential growth typical of exact optimization methods. The key insight from Table 2, therefore, is that the

proposed method achieves its optimization objectives without sacrificing computational efficiency. This is made possible by its two-stage architecture—fast edge clustering followed by lightweight cloud refinement—which ensures that the overall planning time remains within acceptable bounds even for large-scale operations.

The practical value of this efficiency is best understood within the context of real-world deployment. Notably, in high task-to-UAV ratio scenarios, e.g., Scenario 8, the proposed method completes planning within 10.32 s. This is well within the practically acceptable window for real-time deployment in emergency response, where re-planning intervals are typically 5–10 min (we set at 5 min; see 5.4 for detail).

In conclusion, the combination of stable computational efficiency makes the proposed method highly suitable for deployment on resource-constrained edge devices. It enables frequent and responsive replanning without system overload, while still benefiting from cloud-based global optimization when connectivity permits.

(2) Analysis of Total Cost

Table 3 presents the total travel distance, which reflects the overall energy consumption of the mission. Compared with the baseline methods, the proposed method achieves total distances generally comparable to the best-performing approaches, and even outperforms them in Scenarios 9–11. This indicates that the competitive task transfer mechanism effectively reduces overall resource consumption by optimizing insertion costs at the individual task level.

The greedy-based method consistently yields the highest total distances, particularly in Scenarios 4 and 8, due to its myopic allocation that produces locally optimal but globally inefficient assignments. Importantly, this high total distance does not translate into improved load balancing; as shown in Table 4, the greedy method also exhibits the worst makespan, making it inferior for both metrics.

While K-means clustering excels when tasks form compact, isolated groups, and Voronoi diagrams offer fast spatial partitioning, both methods exhibit rigidity in the face of linear or irregular fronts.

A deliberate feature of the proposed method is observed in Scenarios 5–8, where its total distance is slightly higher (by 2–3%) than those of Voronoi and K-means. For instance, in Scenario 6, the proposed method yields 824.43 compared to 805.72 (Voronoi) and 815.47 (K-means). This modest increase stems from the cooperative task transfer mechanism, which prioritizes load balancing over absolute distance minimization by moving tasks from overloaded to underloaded UAVs—even when those tasks are spatially closer to their original assignee. The resulting trade-off is quantified by the efficiency ratio: the percentage reduction in makespan per 1% increase in total distance. In Scenario 12, a 1.53% distance increase relative to K-means yields a 14.4% makespan reduction (ratio 9.4:1); against Voronoi, a 0.60% increase achieves a 6.9% makespan reduction (ratio 11.5:1). In Scenario 8, a 1.27% distance increase relative to K-means results in a 10.5% makespan reduction relative to Voronoi (ratio 8.3:1). These high ratios demonstrate that the cooperative–competitive mechanism efficiently converts marginal additional flight distance into substantial gains in mission timeliness. This design philosophy—prioritizing makespan over absolute distance—is strategically well-suited to emergency response, where faster situational awareness directly supports critical decision-making.

(3) Analysis of Max UAV Cost

Table 4 presents the makespan—the maximum flight distance among all UAVs—which serves as the primary optimization objective of this study, as it directly determines the time required to complete the full surveillance mission. The results demonstrate that the proposed method achieves the lowest makespan in 10 out of 12 scenarios and

remains within 1% of the optimal value in the remaining two, confirming its effectiveness in balancing workloads across the UAV swarm.

The greedy-based method consistently yields the highest makespan, with particularly large gaps in high task-to-UAV ratio scenarios (e.g., Scenario 8 and 12). This is attributed to its myopic allocation, which frequently overloads one or two UAVs. The Voronoi-based and K-means-based methods perform reasonably well in many scenarios, but their makespan values are still notably higher than those of the proposed method. For instance, in Scenario 8, the proposed method reduces makespan by 10.4% compared to Voronoi and by 0.5% compared to K-means; in Scenario 12 (90 tasks, five UAVs), the improvements reach 6.9% and 14.4%, respectively. These gaps arise because Voronoi diagrams produce rigid spatial partitions that may assign disproportionately large areas to some UAVs, while K-means clustering, although more flexible, lacks any cross-cluster load-balancing mechanism.

The superiority of the proposed method stems from two integrated design features. First, the edge-side line clustering heuristic groups tasks along directional axes, producing an initial allocation that is already better aligned with the sequential nature of UAV travel. Second, the cloud-side cooperative–competitive mechanism actively transfers tasks from overloaded to underloaded UAVs; the cooperative phase (Algorithm 3) explicitly targets makespan minimization by incentivizing transfers that reduce the peak load, even at the cost of a slight increase in total travel distance (as documented in Table 3). The efficiency of this trade-off has been quantified: a modest 1–3% increase in total distance yields makespan reductions of 10–15%, with efficiency ratios exceeding 8:1 in several scenarios. Such ratios confirm that the method's prioritization of makespan over absolute distance is both deliberate and advantageous.

In summary, the makespan results validate that the proposed cloud–edge collaborative framework successfully achieves its primary design goal: minimizing the makespan through balanced task allocation. By combining a geometry-aware initial partition with a global load-balancing refinement, the method consistently outperforms existing approaches, particularly as problem scale increases—a critical advantage for large-scale forest fire monitoring.

5.1.2. Analysis of Robustness Evaluation

The robustness experiments reveal how the spatial structure of fire points influences the relative performance of the four algorithms. The key observation is that the advantage of the proposed method is most pronounced under the linear and clustered distribution, and shrinks under uniform random. This trend can be explained by the design philosophy of each algorithm and the nature of the task allocation problem.

When tasks are scattered uniformly at random (as shown in Figure 7), all three benchmark methods perform similarly to each other, with the proposed method holding a slight advantage, whereas the greedy method suffers severely because its myopic allocation consistently overloads one UAV. The performance gap between the proposed method and the second-best method (K-means) is smaller under uniform random distribution than under linear or clustered distributions. This is because the line clustering heuristic no longer matches the actual point distribution; its linear segments become arbitrary, and the algorithm essentially falls back to a centroid-based partition comparable to K-means. Nevertheless, the cooperative transfer phase still manages to reduce the maximum load compare to the second-best method (Figure 7C,D).

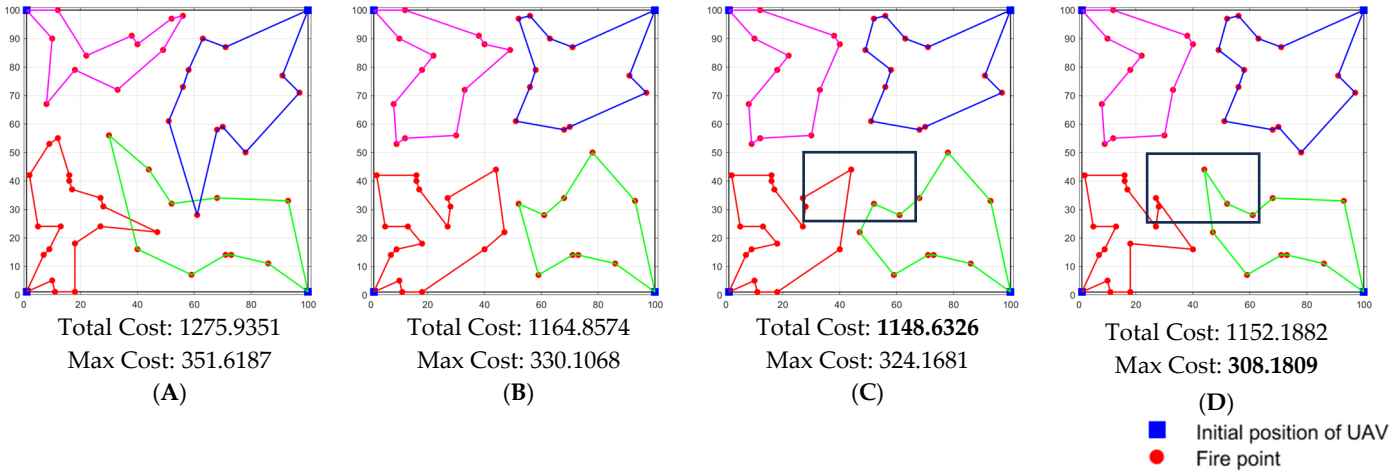

Figure 7. Example of random generation modes: (**A**) greedy based, (**B**) Voronoi diagram based, (**C**) K-means clustering based, (**D**) proposed method.

Under the clustered distribution, the task points naturally form several compact groups, a structure for which the K-means-based method is particularly well suited. As shown in Figure 8C, K-means achieves its best relative performance. The proposed method attains a comparable makespan while also ensuring balanced workloads across UAVs (Figure 8C,D). Although its total distance is higher due to cross-cluster task transfers that reduce peak load, this trade-off aligns with the primary objective of makespan minimization. In contrast, the Voronoi-based method, while capable of achieving some degree of load balancing in moderately scattered scenarios, pays a higher price than the proposed method (Figure 8B,D), indicating that its rigid spatial partitioning is less adaptable to cluster geometries. Notably, under the clustered distribution, the proposed method does not artificially split clusters to achieve perfect balance; instead, it maintains natural groupings while only transferring boundary tasks to reduce peak load. This behavior, visualized in Figure 8D, demonstrates that the algorithm does not enforce "averaging" at the expense of spatial coherence—a property that aligns with the need for preserving task context in priority-aware extensions. A legitimate concern regarding makespan minimization is that it might "average out" responses, delaying attention to the most dangerous fire sectors. Our experimental results alleviate this concern to some extent.

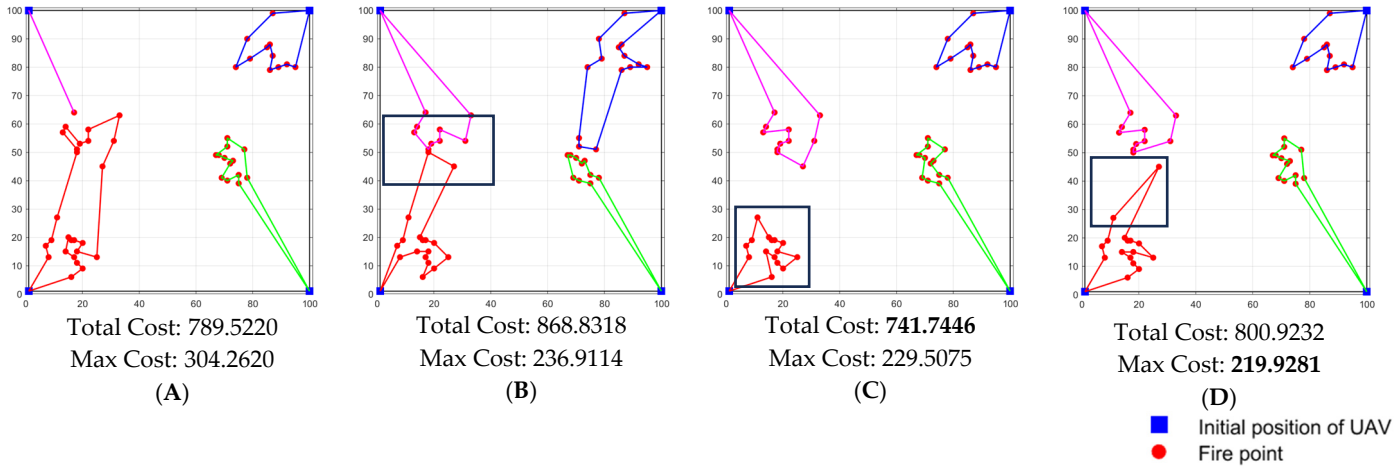

Figure 8. Example of clustered generation modes: (**A**) greedy based, (**B**) Voronoi diagram based, (**C**) K-means clustering based, (**D**) proposed method.

Under the linear generation modes (as shown in Figure 9), which mimics a continuous fire front, the proposed method reduces the makespan by 38.4% compared to the greedy-based method and by about 5–6% compared to Voronoi and K-means. This substantial gain stems from two factors. First, the edge-side line clustering heuristic explicitly models the expected path shape by iteratively fitting line segments, thereby grouping tasks that lie along the same direction. In contrast, the greedy model relies on point-to-point distances and ignores the sequential nature of UAV travel; it tends to create clusters that are spatially compact but may force UAVs to crisscross the fire line (Figure 9A). Second, line clustering effectively captures the edge of the fire front and generates well-structured routes (Figure 9D). Finally, the cloud-side cooperative–competitive mechanism further refines the allocation by transferring tasks between UAVs to balance the loads, a step that the static clustering methods lack. The result is not only a lower makespan but also more coherent, non-intersecting routes that follow the fire front—a desirable property for real-world monitoring where crossing paths would waste energy and complicate coordination.

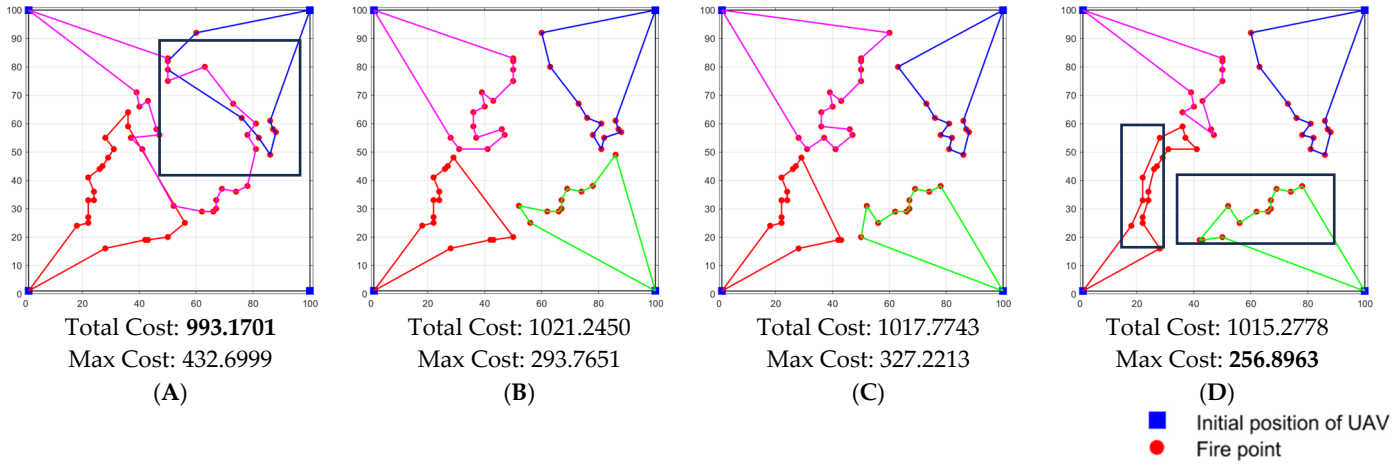

Figure 9. Example of linear generation modes: (**A**) greedy based, (**B**) Voronoi diagram based, (**C**) K-means clustering based, (**D**) proposed method.

Based on the results of the random task experiments and the robustness analysis, the proposed method performs effectively under the line clustering assumption and maintains algorithmic robustness across other task distribution patterns. Its ability to adapt to diverse fire front geometries while preserving load balance and computational efficiency is particularly noteworthy.

5.2. Analysis of Real Forest Fire Simulation Results

The results for the Liangshan case provide concrete evidence of the method's real-world applicability. The proposed method achieves the lowest average computation time (1.7330 s), demonstrating its suitability for low-latency edge deployment. More importantly, it attains the lowest maximum UAV cost (92.9065), outperforming the next best method by over 24.5%. This significant reduction directly validates the effectiveness of the cooperative–competitive mechanism in minimizing the critical makespan. It is noted that this comes with a trade-off of a higher total cost compared to some benchmarks, which is a deliberate design choice favoring load balance and timeliness over absolute distance minimization—a rational strategy for emergency response.

Unlike baseline methods that minimize total flight distance, our cooperative–competitive mechanism explicitly balances the maximum load, which is more aligned with the time-critical nature of fire monitoring. This design choice, however, may lead to

slightly higher total costs in some configurations—a trade-off that is acceptable given the significant gain in makespan reduction.

Compared to other methods, the proposed method achieves the lowest max UAV cost score at the expense of a higher total monitoring cost. This trade-off directly reflects the design intent: through the cooperative task transfer mechanism in the conflict resolution stage, a small increase in total cost is actively accepted to achieve a substantial reduction in maximum load cost. This strategy directly optimizes the most critical metric in emergency scenarios—the execution time of the monitoring plan. It also improves fairness by avoiding long waits for targets assigned to overloaded UAVs. This represents a successful practice of trading controllable total cost for critical timeliness and system robustness, ensuring load-balanced and sustainable full-domain monitoring capability in forest fire emergencies.

And the proposed approach demonstrates superior performance across multiple simulated forest fire emergency scenarios and a real-world case study: the edge-based line clustering task allocation algorithm ensures allocation accuracy while maintaining low computation time; the cloud-based cooperative–competitive conflict resolution mechanism effectively achieves load balancing, significantly reduces the overall monitoring time, and maintains high stability and robustness. The results underscore the effectiveness of the cloud–edge collaborative planning method in balancing computational load and communication demands.

5.3. Analysis of Ablation Study Results

The ablation study provides crucial insights into the contribution of each component and, importantly, allows for an assessment of the method's robustness.

The ablation results in Table 7 and Figure 6 clearly delineate the contribution of each method component. Config. 2 shows that the cloud-based cooperative–competitive mechanism alone can significantly improve both total and max cost from a poor random start. The most critical finding is from Config. 3: the edge-side line clustering algorithm operating in isolation achieves a max UAV cost of 103.09, which is already a 50.4% improvement over the baseline and is highly competitive. This underscores the standalone strength and operational resilience of the edge component, a vital feature for scenarios with compromised cloud connectivity. The Full Method (Config. 4) then combines both components to deliver the optimal balanced performance, further refining the max cost by 9.9%.

Comparing Config. 1 (Random Allocation) and Config. 2 (Random + Cooperative–Competitive) reveals the standalone effect of the conflict resolution mechanism. While the random allocation yields the shortest computation time due to its simplicity, it results in the highest total cost and, most critically, the highest max UAV cost, indicating severe load imbalance. Applying the cooperative–competitive mechanism (Config. 2) significantly improves both the total cost and the max UAV cost, demonstrating its potent capability to optimize a given allocation towards lower cost and better balance, even from a highly suboptimal starting point. However, the paths generated under Config. 2 (as shown in Figure 10) exhibit frequent crossing into each other's nominal areas, leading to longer and less efficient overall paths. This indicates that the mechanism, while effective, performs local optimizations and is constrained by the poor global structure of the initial random allocation.

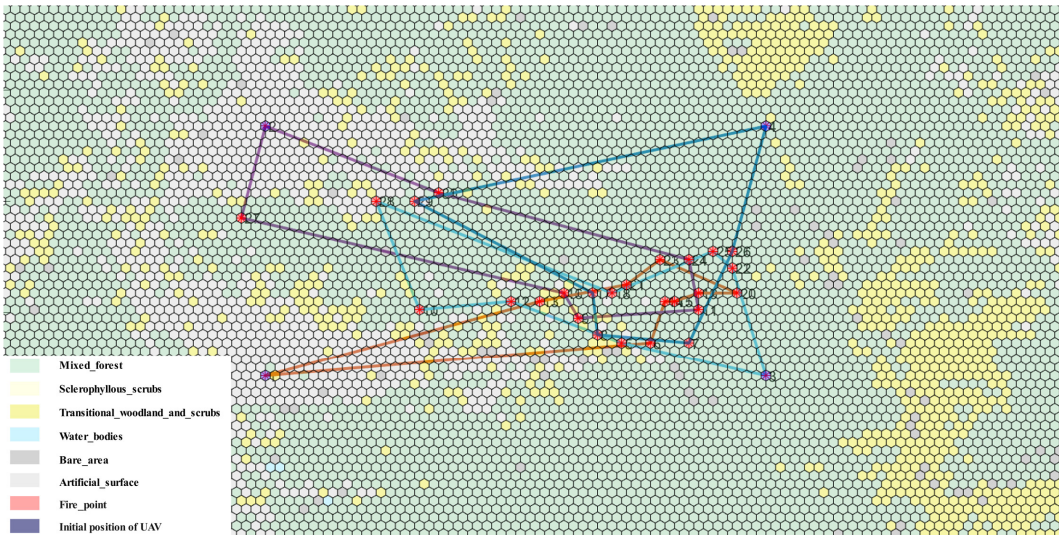

Figure 10. Visualization of task paths for Config. 2 (Random Initial Allocation + Cooperative–Competitive Mechanism).

This result of Config. 3 (Only Line Clustering) is critical for evaluating the method's applicability in real-world emergency scenarios, where persistent and reliable connectivity between the edge and cloud cannot be guaranteed. It demonstrates that the core innovation of this work—the line clustering algorithm—is not merely a preprocessing step but a powerful standalone planning tool. Our simulation studies incorporating communication degradation models (e.g., packet loss and variable latency) further substantiate this point. The edge-generated solution provides a high-quality, workload-balanced plan (as evidenced by its low max UAV cost in the ablation study) that ensures the mission can proceed robustly and efficiently even with degraded or intermittent cloud connectivity. This ability to maintain core functionality under communication constraints is a key architectural advantage. It contrasts with purely distributed methods, whose negotiation-heavy protocols suffer under packet loss and latency, and purely centralized methods, which become inoperable without a stable connection.

Finally, the Full Method (Config. 4) represents the optimal case with full connectivity. It leverages the strong foundation provided by the edge allocation and uses cloud resources to perform a fine-tuning optimization, primarily focused on load balancing, which yields a further 9.9% reduction in the max UAV cost.

In summary, the proposed cloud–edge collaborative planning method offers a flexible and robust solution: it delivers a superior, globally optimized plan when cloud connectivity is available, and in the event of a cloud–edge communication failure, the edge component alone can provide a high-quality, executable plan, ensuring the fundamental operability of the edge computing system under adverse conditions.

5.4. On the Scheduling of Re-Planning in Dynamic Environments

The frequency of re-planning invocations in dynamic fire environments should be informed by empirical fire behavior data. According to field-based fire behavior studies [33], the 10% wind speed rule of thumb for severe burning conditions suggests that under extreme winds (>30 km/h) with critically dry fuels, spread rates can reach 0.8–12.5 km/h.

Based on these empirical values, our choice of a 5 min re-planning interval and 1 km displacement threshold is conservatively designed. At a low spread rate of 0.8 km/h, a fire front would advance approximately 66 m in 5 min, well below our 1 km threshold. Even at the extreme high percentile rate of 12.5 km/h, the 5 min advance would be 1041 m—just over the 1 km threshold. The 1 km threshold therefore captures significant fire progression

while avoiding excessive re-planning triggered by minor movements. When fire rapidly spreads, which would require more frequent updates, our event-triggered mechanism naturally adapts to such conditions by invoking re-planning when the accumulated advance exceeds the threshold, regardless of elapsed time.

This parameterization balances adaptability with computational efficiency, as the edge-side line clustering completes in around 10 s, even for 90 tasks, enabling frequent re-planning without system overload.

5.5. Comprehensive Advantages of the Proposed Method

The superior performance in Max Single-UAV Cost, which is our primary optimization objective, is achieved through the synergistic effect of our method: the edge-side line clustering provides a well-balanced initial allocation by considering path-like structures, and the cloud-side task transfer mechanism (Algorithms 2 and 3) not only optimizes task costs but also actively redistributes tasks from overloaded to underloaded UAVs, thereby reducing peak workloads even if this slightly increases the total travel distance. This is evident in the real-world case, where our method achieves the lowest max cost despite a higher total cost.

The marginally higher total travel distance of the proposed method is a deliberate design choice: the cooperative transfer phase intentionally trades a small increase in total cost for a substantial reduction in makespan. This trade-off is Pareto-optimal for time-critical emergency response, where faster situational awareness outweighs marginal energy savings. The balance can be tuned by weighting the cooperative gain term in Equation (14). Our choice emphasizes timeliness, suitable for rapid assessment scenarios.

6. Conclusions

This study presents a cloud–edge collaborative method that translates advances in distributed computing into a practical strategy for enhancing the effectiveness of UAV-based forest fire monitoring. By decoupling rapid, fire spread-informed task allocation at the edge from global load-balancing optimization in the cloud, the method provides incident commanders with a tool for orchestrating UAV resources that delivers more timely and equitable situational awareness of the fire front.

Validation across simulated scenarios and the 2020 Liangshan fire case demonstrates that this strategy can reduce the time to complete a full situational assessment (makespan) by up to 24.5% compared to conventional deployment methods. This reduction directly compresses the decision-making cycle for initial attack. Furthermore, the edge-centric design ensures operational robustness, maintaining high-quality surveillance even under constrained communication—a critical feature for remote fire operations.

The current line clustering heuristic is best suited to fires with discernible linear fronts; in the future we will integrate dynamic fire spread prediction models and digital elevation data for adaptive planning in complex terrains, incorporating fire priority to optimize resource allocation. Extending this method to coordinate heterogeneous platforms (e.g., satellites, ground sensors) and leveraging next-generation communication protocols will be key steps towards building a more resilient, multi-modal intelligent monitoring system for forest fire management.

Author Contributions: Conceptualization, Y.D.; methodology, Y.D.; software, Y.D.; validation, P.L.; formal analysis, P.L.; investigation, X.G.; resources, X.G.; data curation, X.G.; writing—original draft preparation, Y.D.; writing—review and editing, Z.L.; visualization, Y.D.; supervision, Z.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the Key Research Program of the Chinese Academy of Sciences, grant number KGFZD-145-24-03-03.

Data Availability Statement: The original contributions presented in the study are included in the article; further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Abdusalomov, A.; Umirzakova, S.; Kutlimuratov, A.; Mirzaev, D.; Dauletov, A.; Botirov, T.; Zakirova, M.; Mukhiddinov, M.; Cho, Y.I. Lightweight UAV-Based System for Early Fire-Risk Identification in Wild Forests. *Fire* **2025**, *8*, 288. [CrossRef]
2. Filkov, A.I.; Holyland, B.; Cirulis, B.; Moinuddin, K.; Sutherland, D.; Sharples, J.; Hilton, J.; Clements, C.B.; Penman, T.D. Investigating the dynamic behaviour of merging fire fronts. *Int. J. Wildland Fire* **2025**, *34*, WF24126. [CrossRef]
3. Sun, J.; Ma, B.; Wu, Z.; Yang, X.; Wu, T.; Chen, P. Joint optimization of UAV swarm path planning and task allocation balance in earthquake scenarios. *J. Comput. Appl.* **2024**, *44*, 3232–3239.
4. Viseras, A.; Meissner, M.; Marchal, J. Wildfire Front Monitoring with Multiple UAVs Using Deep Q-Learning. *IEEE Access* **2025**, *13*, 123269–123281. [CrossRef]
5. Xu, Y.; Huang, D.; Zhang, L.; Zhang, F. Improved Multi-Objective Crested Porcupine Optimizer for UAV Forest Fire Cruising Strategy. *Fire* **2026**, *9*, 40. [CrossRef]
6. Liu, Z.; Shi, Z.C.; Liu, W.; Zhang, L.; Wang, R. Integrated Optimization of Ground Support Systems and UAV Task Planning for Efficient Forest Fire Inspection. *Drones* **2025**, *9*, 684. [CrossRef]
7. Cai, W.; Wang, D.; Li, W.; Chen, Z.; Pang, Y.; Chen, S.; Zhang, Y. Method for Continuously Monitoring of Forest Fire in i.e., Forest Cruise Unmanned Aerial Vehicle, Involves Fire Automatically Reporting Department Report by Ground Station System that Location of Unmanned Aerial Vehicle is at Location of Fire. CN114283548-A, 27 December 2021.
8. Dasdemir, E.; Jose, E.; Batta, R. Scheduling and routing with degradation-triggered job arrivals: An application to forest firefighting with an unmanned aerial vehicle fleet. *Eur. J. Oper. Res.* **2026**, *330*, 715–732. [CrossRef]
9. Zhang, M.H.; Wang, H.L.; Lun, Y.B.; Yang, Z.Y.; Wang, Y.X. Multi-UAV Efficient Cooperative Forest Fire Patrol in 3-D Mountain Environments. *IEEE Trans. Aerosp. Electron. Syst.* **2025**, *61*, 15737–15756. [CrossRef]
10. Hamza, A.; Darwish, A.H.; Rihawi, O. A new local search for the bees algorithm to optimize multiple traveling salesman problem. *Intell. Syst. Appl.* **2023**, *18*, 200242. [CrossRef]
11. Elhenawy, M.; Abutahoun, A.; Alhadidi, T.I.; Jaber, A.; Ashqar, H.I.; Jaradat, S.; Abdelhay, A.; Glaser, S.; Rakotonirainy, A. Visual Reasoning and Multi-Agent Approach in Multimodal Large Language Models (MLLMs): Solving TSP and mTSP Combinatorial Challenges. *Mach. Learn. Knowl. Extr.* **2024**, *6*, 1894–1920. [CrossRef]
12. Mahmoudinazlou, S.; Kwon, C. A hybrid genetic algorithm for the min-max Multiple Traveling Salesman Problem. *Comput. Oper. Res.* **2024**, *162*, 106455. [CrossRef]
13. Akshya, J.; Priyadarsini, P.L.K. Graph-based path planning for intelligent UAVs in area coverage applications. *J. Intell. Fuzzy Syst.* **2020**, *39*, 8191–8203. [CrossRef]
14. Smaili, F. A hybrid genetic-simulated annealing algorithm for multiple traveling salesman problems. *Decis. Sci. Lett.* **2024**, *13*, 709–728. [CrossRef]
15. Gasteratos, G.; Karydis, I. Efficient Drone Data Collection in WSNs: ILP and mTSP Integration with Quality Assessment. *World Electr. Veh. J.* **2025**, *16*, 560. [CrossRef]
16. Tang, H.L.; Lian, X.R.; Yu, C.; Zhang, T.; Liu, J. DOUBLESQUEEZE: Parallel Stochastic Gradient Descent with Double-pass Error-Compensated Compression. In Proceedings of the 36th International Conference on Machine Learning (ICML), Long Beach, CA, USA, 9–15 June 2019.
17. Ling, Z.X.; Zhou, Y.L.; Zhang, Y. Solving multiple travelling salesman problem through deep convolutional neural network. *IET Cyber-Syst. Robot.* **2023**, *5*, e12084. [CrossRef]
18. Ma, S.; Ruan, J.Q.; Du, Y.L.; Bucknall, R.; Liu, Y.C. An End-to-End Deep Reinforcement Learning Based Modular Task Allocation Framework for Autonomous Mobile Systems. *IEEE Trans. Autom. Sci. Eng.* **2025**, *22*, 1519–1533. [CrossRef]
19. Huang, J.; Du, B.H.; Zhang, Y.M.; Quan, Q.; Wang, B.; Mu, L.X. A Pesticide Spraying Mission Allocation and Path Planning with Multicopters. *IEEE Trans. Aerosp. Electron. Syst.* **2024**, *60*, 2277–2291. [CrossRef]
20. Bai, Y.F.; Lindqvist, B.; Nordström, S.; Kanellakis, C.; Nikolakopoulos, G. Cluster-based Multi-robot Task Assignment, Planning, and Control. *Int. J. Control Autom. Syst.* **2024**, *22*, 2537–2550. [CrossRef]
21. Luo, J.; Su, Y.M. Path planning for Multi-USV target coverage in complex environments. *Ocean Eng.* **2024**, *312*, 119090. [CrossRef]
22. Xiong, C.K.; Chen, D.F.; Lu, D.; Zeng, Z.; Lian, L. Path planning of multiple autonomous marine vehicles for adaptive sampling using Voronoi-based ant colony optimization. *Robot. Auton. Syst.* **2019**, *115*, 90–103. [CrossRef]

23. Zhu, P.; Jiang, S.; Zhang, J.; Xu, Z.; Sun, Z.; Shao, Q. Multi-Target Firefighting Task Planning Strategy for Multiple UAVs Under Dynamic Forest Fire Environment. *Fire* **2025**, *8*, 61. [CrossRef]
24. Chen, X.Y.; Zhang, P.; Du, G.L.; Li, F. A distributed method for dynamic multi-robot task allocation problems with critical time constraints. *Robot. Auton. Syst.* **2019**, *118*, 31–46. [CrossRef]
25. Tan, C.R.; Liu, X. Improved two-stage task allocation of distributed UAV swarms based on an improved auction mechanism. *Int. J. Mach. Learn. Cybern.* **2024**, *15*, 5119–5128. [CrossRef]
26. Chen, X.Y.; Zhang, P.; Li, F.; Du, G.L.; IEEE. A cluster first strategy for distributed multi-robot task allocation problem with time constraints. In Proceedings of the 1st World Robot Conference (WRC)/Symposium on Advanced Robotics and Automation (SARA), Beijing, China, 16 August 2018; pp. 102–107.
27. Dong, Y.M.; Zhang, J.; Xie, C.Z.; Li, Z.Y. A Survey of Key Issues in Edge Intelligent Computing Under Optimization and Computing Offloading. *J. Electron. Inf. Technol.* **2024**, *46*, 765–776. [CrossRef]
28. Zheng, F.; Zhu, D.; Zang, W.; Yang, J.; Zhu, G. Edge Computing: Review and Application Research on New Computing Paradigm. *J. Front. Comput. Sci. Technol.* **2020**, *14*, 541–553.
29. Qi, X.; Yuan, Y.; Zhou, J.; Li, Z.; Zhou, Z. Comparative analysis of forest fire spread simulation and reinitialization methods based on FARSITE and Cell2Fire. *Natl. Remote Sens. Bull.* **2025**, *29*, 945–957. [CrossRef]
30. Singh, H.; Ang, L.; Lewis, T.; Paudyal, D.; Acuna, M.; Srivastava, P.K.; Srivastava, S.K. Trending and emerging prospects of physics-based and ML-based wildfire spread models: A comprehensive review. *J. For. Res.* **2024**, *35*, 135. [CrossRef]
31. Dorigo, M.; Birattari, M.; Stützle, T. Ant colony optimization: Artificial ants as a computational intelligence technique. *IEEE Comput. Intell. Mag.* **2006**, *1*, 28–39. [CrossRef]
32. Zhao, W.Q.; Meng, Q.G.; Chung, P.W.H. A Heuristic Distributed Task Allocation Method for Multivehicle Multitask Problems and Its Application to Search and Rescue Scenario. *IEEE Trans. Cybern.* **2016**, *46*, 902–915. [CrossRef]
33. Cruz, M.G.; Alexander, M.E.; Fernandes, P.M.; Kilinc, M.; Sil, Â. Evaluating the 10% wind speed rule of thumb for estimating a wildfire's forward rate of spread against an extensive independent set of observations. *Environ. Model. Softw.* **2020**, *133*, 104818. [CrossRef]